The Three-Question Framework for Every Function - for the paper on Breast Cancer Detection

Step 1: Load the Dataset

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`pd.read_csv()`	Parses raw text from a delimited file into a two-dimensional, labeled tabular structure (DataFrame).	Ingests `data.csv` so diagnostic variables and target labels are accessible as structured arrays.	Downstream tabular operations, indexing, and linear algebra pipelines cannot run on raw text files.
`head()`	Returns the first N contiguous records (0 to $N-1$) of the DataFrame.	Confirms whether feature columns, target strings (`M` / `B`), and numeric values loaded under proper headers.	Parsing anomalies (such as misaligned header rows) remain undetected before preprocessing begins.
`shape`	Returns the dimensional tuple (N, M) representing matrix cardinality (N rows, M features).	Verifies the expected volume of samples (569) and original columns (33 including metadata).	Dimension mismatches or truncated data loads pass silently into model pipelines.
`.columns`	Returns an immutable index sequence containing the exact string labels of every column.	Identifies specific feature labels and exposes artifacts such as `Unnamed: 32` or `id`.	Typos in column selection break subsequent programmatic feature filtering.
`info()`	Aggregates non-null entry tallies and inspects memory footprints and byte-level data types.	Confirms whether features imported as floating-point numbers or were corrupted into string objects.	Numeric features parsed as `object` types will cause statistical scaling and mathematical estimators to crash.

Step 2: Inspect Data Quality

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`isnull()`	Maps elements via a boolean indicator: $\mathrm{I}(x = \mathrm{NaN})$.	Flags missing measurements across nuclear morphometry fields before feeding matrices to estimators.	Unidentified missing elements cause linear algebra and matrix operations in scikit-learn to fail.
`sum()`	Evaluates the discrete sum: $\sum_{i=1}^{N} \mathrm{I}(x_i = \mathrm{NaN})$ across axes.	Quantifies the volume of missingness per feature to decide whether imputation or column dropping is required.	Unchecked missing counts obscure whether an entire feature column lacks valid data.

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`duplicated()`	Computes row-wise hash equivalence to flag identical feature tuples.	Verifies whether patient biopsy records were accidentally recorded more than once.	Duplicate instances artificially inflate evaluation scores and create train-test leakage.
`drop_duplicates()`	Filters out identical coordinate vectors, retaining only unique sample observations.	Ensures model parameters fit strictly over distinct medical cases without redundant weighting.	Repeated patient samples bias variance calculations and violate sample independence assumptions.
`.describe()`	Computes the five-number summary plus mean, standard deviation, and sample counts.	Exposes drastic scale differences between metrics (e.g., `area_mean` > 1000 vs. `smoothness_mean` < 0.2).	Scale variations go unnoticed, masking the necessity of feature normalization.

Step 3: Define Features and Target

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`drop()`	Computes projection $X = D \setminus \{c_j\}$, removing target and metadata vectors from the feature space.	Removes non-predictive `id` columns and separates target labels `diagnosis` from input features.	Keeping `id` allows models to memorize patient identity numbers rather than tumor geometry.
`.map()`	Applies a discrete bijection: $f(\text{'B'}) = 0, f(\text{'M'}) = 1$.	Maps categorical string diagnoses to numerical scalars required by binary loss functions.	Loss functions expecting numerical targets cannot evaluate string labels.
`.replace()`	Direct key-value lookup and substitution across values in a targeted Series.	Serves as an alternative approach to replace categorical labels with binary values (0/1).	Retaining string labels leaves targets in an uncomputable format for optimization solvers.

Step 4: Perform the Train-Test Split First

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`train_test_split()`	Partitions index set $\{1, \dots, N\}$ into disjoint sets I_{train} and I_{test} given split ratio α.	Allocates unseen data (e.g., 20%) to evaluate model generalization on new biopsy samples.	Splitting after preprocessing causes data leakage, artificially inflating validation metrics.
`stratify=y`	Enforces equal conditional class probabilities: $P_{\text{train}}(y = c) = P_{\text{test}}(y = c)$.	Benign cases outnumber malignant cases (~63% vs ~37%); stratification maintains this ratio in both sets.	A standard random split can undersample malignant cases in the test set, skewing recall evaluation.
`random_state`	Fixes the seed for the pseudorandom number generator algorithm.	Guarantees identical sample indices in splits across laboratory runs for fair model comparison.	Inconsistent splits make it impossible to determine if improvements stem from model tuning or split luck.

Step 5: Handle Missing Values

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`SimpleImputer()`	Implements univariate replacement via statistic θ (e.g., mean $\bar{x} = \frac{1}{n}\sum x_i$).	Fulfills the standard clinical data preparation protocol defined in the study.	Missing values break scikit-learn estimators during gradient steps or distance evaluations.
`fit()`	Calculates column-wise statistics θ_{train} strictly using X_{train}.	Restricts baseline statistical parameters to training observations only.	Calculating parameters on the full dataset leaks test distribution information into the training phase.
`transform()`	Imputes missing entries: $x_{ij} \leftarrow \theta_j$ using the previously learned θ_j.	Fills test set gaps using training parameters to simulate real-world inference conditions.	Recalculating mean statistics over test samples leaks test information into evaluation.
`fit_transform()`	Computes θ_{train} and applies it to X_{train} in a single combined call.	Streamlines the preprocessing step on the training set before separate test transformation.	Calling it on the test set recalculates new parameters rather than applying training statistics.

Step 6: Scale Numerical Features

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`MinMaxScaler()`	Applies the linear transformation: $x' = \frac{x - \min(x)}{\max(x) - \min(x)}$.	Rescales disparate nuclear measurements (e.g., `area` vs `smoothness`) to the unit interval $[0, 1]$.	Distance and margin-based models (KNN, SVM, LR) become dominated by large-scale features.
`fit()`	Computes and stores extrema vectors: $\mathbf{x}_{\min}, \mathbf{x}_{\max}$ from X_{train}.	Learns the bounding range strictly from training patient data.	Fitting on test samples incorporates unseen distribution boundaries into model preparation.
`transform()`	Scales an input matrix using previously stored training extrema: $\frac{X - \mathbf{x}_{\min}}{\mathbf{x}_{\max} - \mathbf{x}_{\min}}$.	Normalizes the untouched test set using the reference scales established during training.	Re-fitting a scaler on test data artificially compresses or shifts out-of-distribution values.
`fit_transform()`	Learns the training boundaries and scales X_{train} simultaneously.	Efficiently scales training matrices prior to feature selection or model fitting.	Applying this to test data fits a new scaler, introducing test leakage.

Step 7: Perform Exploratory Data Analysis

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`value_counts()`	Computes discrete sample frequencies: $n_c = \sum \mathrm{I}(y_i = c)$.	Measures class balance between benign (357) and malignant (212) cases.	Unidentified class imbalance leads to unrepresentative accuracy metrics hiding high false negative rates.
`sns.countplot()`	Renders vertical bar charts parameterized by class	Visually demonstrates target distribution to guide evaluation	Without visual checks, subtle class imbalances are easily overlooked

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
	occurrences.	metric choices.	during pipeline design.
`sns.violinplot()`	Combines box-plot quartiles with mirrored kernel density estimation curves.	Highlights distribution spread and multimodal separation across malignant vs benign tumors.	Skewness and multimodal feature behavior remain hidden when inspecting only means and standard deviations.
`sns.boxplot()`	Graphically plots medians, interquartile ranges (IQR), and whisker thresholds.	Identifies extreme outliers in nuclear feature distributions like `area_worst`.	Unchecked anomalous outliers distort regression coefficients and linear decision boundaries.
`sns.histplot()`	Partitions continuous feature values into discrete bins to estimate probability density.	Evaluates whether individual morphometric features follow a normal Gaussian distribution.	Violations of normality assumptions go unnoticed before applying parametric models like Gaussian Naïve Bayes.
`.corr()`	Calculates the Pearson coefficient: $r_{xy} = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y}$.	Maps linear collinearities between geometric measurements (such as `radius`, `perimeter`, and `area`).	Undetected collinearities inflate coefficient variance and destabilize linear models.
`sns.heatmap()`	Maps a two-dimensional matrix of correlation values onto a diverging color spectrum.	Provides a visual map of multicollinear feature clusters across all 30 variables.	Redundant, co-dependent features cannot be easily targeted for correlation-based pruning.

Step 8: Correlation-Based Feature Selection

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`.corr()`	Builds a symmetric $p \times p$ correlation matrix over training features.	Provides pairwise dependencies needed to apply the $	r_{ij}
`np.triu()`	Sets elements below the main diagonal to zero: $A_{ij} = 0 \quad \forall\, i > j$.	Prevents processing symmetric feature pairs twice and excludes diagonal self-correlations ($r = 1.0$).	Without an upper-triangle mask, drop loops remove both features in a correlated pair instead of just one.
`.drop()`	Removes specified collinear feature columns from the feature matrix.	Discards redundant geometric features (e.g., dropping `perimeter` if correlated with `radius`).	Severe multicollinearity destabilizes models such as Logistic Regression.

Step 9: Chi-Square Feature Selection

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`chi2()`	Computes independence test statistic: $\chi^2 = \sum \frac{(O-E)^2}{E}$.	Evaluates whether categorical tumor diagnosis is statistically dependent on feature values.	Negative or unscaled values violate chi-square non-negativity assumptions and cause computation errors.

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`SelectKBest()`	Ranks features by test score and selects the top k dimensions.	Filters the feature set down to the k most informative features relative to diagnosis.	Retaining uninformative features adds noise and increases dimensionality.
`fit()`	Computes χ^2 statistics and p-values across X_{train} and y_{train}.	Learns feature importance rankings strictly from training data.	Fitting on the full dataset lets test class associations influence feature selection.
`transform()`	Truncates feature matrices to only the selected top k columns.	Applies the learned feature mask consistently across training and test matrices.	Column mismatches between splits will cause model evaluation to fail.

Step 10: Recursive Feature Elimination (RFE)

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`RFE()`	Backward elimination pruning the lowest-ranked features iteratively based on model weights.	Identifies a compact, high-performing feature subset by accounting for feature interactions.	Manual trial-and-error feature selection is computationally inefficient and prone to bias.
`fit()`	Iteratively trains the base estimator and prunes bottom features on training data.	Derives the optimal feature subset strictly from training samples.	Running RFE on the full dataset allows test patterns to influence feature retention.
`support_`	Returns an array of boolean flags: [True if retained, False if pruned].	Identifies which diagnostic variables to select for downstream model evaluation.	Models cannot isolate and train on the reduced feature subset without the selection mask.
`ranking_`	Assigns an elimination order rank to each feature (1 indicates selected features).	Provides visibility into which tumor metrics are least predictive.	Relative feature utility remains unknown when reviewing pruned variables.

Step 11: Recursive Feature Elimination with Cross-Validation (RFECV)

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`RFECV()`	Automates RFE feature elimination by evaluating candidate subsets using K-fold cross-validation.	Dynamically selects the optimal feature count instead of requiring a manual guess for k.	Arbitrarily setting k risks retaining noisy features or discarding valuable predictors.
`.fit()`	Runs RFE across internal cross-validation training folds to find peak performance.	Validates feature stability without exposing the holdout test set.	Tuning the feature count against test data invalidates final performance estimates.
`.support_`	Returns the boolean mask for the subset that achieved the best cross-validation score.	Filters both training and test matrices to the cross-validated feature set.	The optimal feature subset cannot be extracted or transferred to downstream models.

Step 12: Random Forest Feature Importance

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`RandomForestClassifier()`	Fits an ensemble of bootstrap-aggregated decision trees with random feature subsampling.	Captures non-linear relationships and feature interactions among tumor metrics.	Purely linear analyses miss non-linear diagnostic thresholds.
`fit()`	Optimizes tree split thresholds using training samples and target labels.	Fits ensemble rules solely over training data.	Exposing test data to the forest during training creates data leakage.
`feature_importances_`	Computes normalized mean Gini impurity decrease: $I_j = \frac{1}{N}\sum_{t=1}^{N}\Delta G_t(j)$.	Quantifies the diagnostic utility of each cell nucleus measurement.	Feature selection becomes unguided without numerical importance scores.

Step 13: Extra Trees Feature Importance

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`ExtraTreesClassifier()`	Trains an ensemble of extremely randomized trees with random split thresholds.	Provides an alternative tree-based importance profile with reduced variance.	Relying on a single tree model risks inheriting its specific splitting biases.
`fit()`	Builds randomized decision trees using training observations.	Learns tree split criteria exclusively from training data.	Fitting on full data contaminates the feature selection process with test set signals.
`feature_importances_`	Calculates relative importance scores from variance and impurity reductions across trees.	Supplies the Extra Trees vote for the ensemble feature selection phase.	Cannot contribute to the comparative voting selection step without importance scores.

Step 14: L1-Based Feature Selection

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`LogisticRegression(penalty='l1')`	Optimizes cross-entropy with a Lasso penalty: $\min \left(\mathcal{L} + \lambda \sum	w_j	\right)$.
`SelectFromModel()`	Evaluates model coefficients and retains features exceeding a threshold (e.g., $w_j \neq 0$).	Automatically extracts features with non-zero weights from the trained linear model.	Extracting non-zero coefficients manually requires error-prone indexing.
`.coef_`	Exposes the learned weight vector $\mathbf{w}$ from the linear decision boundary.	Indicates whether a feature correlates positively or negatively with tumor malignancy.	Feature effect directions remain uninterpretable without model coefficients.

Step 15: Voting-Based Feature Selection

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`pd.DataFrame()`	Compiles disparate 1D boolean selection arrays into a unified tabular structure.	Aggregates feature selections across Chi2, RF, ET, RFE, RFECV, and L1 into a single table.	Comparing selection agreement across methods is difficult without a unified table.
`sum(axis=1)`	Evaluates row-wise votes: $V_j = \sum_{m=1}^{M} \mathrm{I}(\mathrm{Selected}_{m,j})$.	Calculates consensus vote totals for each of the 30 candidate features.	Cannot rank features by multi-method agreement without vote aggregation.
`sort_values()`	Sorts records in descending order according to vote tallies.	Isolates the most consistently selected features (e.g., the top 16 features in Table I).	Selecting an arbitrary feature subset ignores consensus across methods.

Step 16: Construct Feature-Selected Datasets

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`transform()`	Applies the learned selection mask to reduce matrix dimensionality from p to k.	Filters feature matrices to the selected subsets across both training and test sets.	Dimension mismatches between training and test sets will cause estimators to crash.
`loc[]`	Label-based indexing to extract specific column subsets: `df.loc[:, selected_cols]`.	Subsets DataFrames while preserving feature names for interpretability.	Integer indexers can misalign columns if feature ordering shifts.
`.columns`	Retrieves string identifiers from the filtered DataFrame.	Verifies that identical features are retained in both splits in matching order.	Misaligned column orders cause models to evaluate incorrect feature inputs silently.
`support_`	Provides the underlying boolean mask from a fitted selector object.	Transfers the training-derived feature subset directly to the test matrix.	Retraining selectors on test data introduces data leakage and can change the selected features.

Step 17: Train Supervised Machine-Learning Models

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`LogisticRegression()`	Fits a linear model for log-odds: $\ln \frac{p}{1-p} = \mathbf{w}^T \mathbf{x} + b$.	Serves as an interpretable linear baseline that resists overfitting on small diagnostic datasets ($N = 569$).	Skipping linear models overlooks simple solutions that may outperform complex ensembles.
`RandomForestClassifier()`	Averages predictions across randomized decision trees: $\frac{1}{B} \sum T_b(\mathbf{x})$.	Captures complex non-linear feature interactions without requiring explicit manual transformations.	High tree counts without depth limits can overfit small training datasets.
`SVC()`	Maximizes the margin separating hyperplane: $\min \frac{1}{2}\lvert\mathbf{w}\rvert^2 + C \sum \xi_i$.	Constructs effective non-linear decision boundaries via kernel transformations.	Highly sensitive to unscaled features, leading to poor boundary optimization if unnormalized.

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`ExtraTreesClassifier()`	Builds an ensemble of extremely randomized trees with random split thresholds.	Reduces model variance further through randomized threshold selection.	Overly randomized splits may underfit informative features if hyperparameters are untuned.
`KNeighborsClassifier()`	Predicts class labels via majority vote among the k nearest Euclidean neighbors.	Serves as a non-parametric baseline that makes no assumptions about data distributions.	Distance calculations become distorted if features are unscaled, degrading accuracy.
`DecisionTreeClassifier()`	Recursively partitions feature space to maximize purity (e.g., Gini impurity or entropy).	Provides fully interpretable, rule-based diagnostic paths for clinical review.	Unconstrained trees overfit small datasets and generalize poorly to unseen patients.
`GaussianNB()`	Applies Bayes' theorem assuming class-conditional feature independence: $P(X \mid y) = \prod P(x_i \mid y)$.	Provides a fast probabilistic baseline that performs well even with limited training data.	Severe multicollinearity violates feature independence assumptions, distorting probability estimates.
`.fit()`	Solves the mathematical objective function over training data $(X_{\text{train}}, y_{\text{train}})$.	Optimizes internal model parameters (weights, thresholds, decision boundaries).	Models cannot generate predictions on test data without first running optimization.

Step 18: Hyperparameter Optimization

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`GridSearchCV()`	Exhaustively evaluates model performance across a predefined parameter grid via cross-validation.	Identifies optimal hyperparameters (e.g., regularization strength C, tree depth) to prevent overfitting.	Default hyperparameters often yield suboptimal bias-variance tradeoffs.
`RandomizedSearchCV()`	Evaluates a fixed budget of random parameter configurations sampled from distributions.	Efficiently searches broad hyperparameter spaces at a fraction of grid search cost.	Exhaustive grid searches over large hyperparameter spaces can be computationally prohibitive.
`StratifiedKFold()`	Partitions data into K folds while preserving class balance in each fold.	Ensures each cross-validation split mirrors the ~37% malignant class distribution.	Unstratified folds can produce validation splits lacking sufficient malignant cases for reliable evaluation.

Step 19: Generate Test Predictions

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`predict()`	Applies the decision rule: $\hat{y} = \mathrm{I}(P(y = 1 \mid \mathbf{x}) \geq 0.5)$.	Assigns discrete diagnostic labels (Benign vs Malignant) to test samples.	Predictions cannot be directly scored against true test labels without discrete classifications.

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`predict_proba()`	Computes calibrated class probability estimates: $P(y = 1 \mid \mathbf{x}) \in [0, 1]$.	Required to sweep decision thresholds when computing ROC and Precision-Recall curves.	Standard discrete predictions cannot be used to plot continuous threshold-based performance curves.
`decision_function()`	Computes the signed distance to the separating hyperplane: $\mathbf{w}^T \mathbf{x} + b$.	Provides continuous confidence scores for margin-based classifiers (e.g., SVM).	Classifiers lacking probability calibration cannot produce continuous evaluation curves without decision scores.

Step 20: Model Evaluation and Required Results

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
`accuracy_score()`	Computes: $\frac{TP+TN}{TP+TN+FP+FN}$.	Measures the overall proportion of correct diagnoses across the test set.	High accuracy can mask high false negative rates in imbalanced datasets.
`precision_score()`	Computes: $\frac{TP}{TP+FP}$.	Measures the proportion of predicted malignant diagnoses that were truly malignant.	Low precision leads to high false alarm rates and unnecessary follow-up procedures.
`recall_score()`	Computes: $\frac{TP}{TP+FN}$.	Quantifies sensitivity: the model's ability to identify all true malignant tumors.	Low recall means malignant tumors go undetected, which carries severe clinical consequences.
`f1_score()`	Computes the harmonic mean: $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$.	Provides a balanced single metric when both false positives and false negatives carry significant costs.	Relying solely on precision or recall encourages optimizing one at the severe expense of the other.
`classification_report()`	Compiles precision, recall, F1-score, and support counts across each target class.	Summarizes per-class performance for both Benign and Malignant categories simultaneously.	Inspecting only overall metrics can hide poor performance on the minority class.
`confusion_matrix()`	Computes the 2×2 contingency table of counts: $[[TN, FP], [FN, TP]]$.	Discloses the exact distribution of diagnostic errors between false alarms and missed cases.	High-level summary metrics conceal the specific types of errors the model makes.
`ConfusionMatrixDisplay()`	Generates a labeled, color-coded grid visualization of the confusion matrix.	Provides an intuitive visual display of diagnostic classification performance.	Raw numerical contingency arrays are less intuitive to interpret during error analysis.
`roc_curve()`	Computes (FPR, TPR) coordinate pairs across varying classification thresholds.	Maps the tradeoff between sensitivity and false alarm rates across decision thresholds.	Evaluating only the default 0.5 threshold obscures diagnostic performance across other operating points.
`RocCurveDisplay()`	Plots the ROC curve relative to a random chance baseline ($\text{AUC} = 0.5$).	Provides visual confirmation of class discrimination across all potential operating points.	Threshold-dependent tradeoffs cannot be visually diagnosed without an ROC curve plot.
`roc_auc_score()`	Evaluates the integral under the ROC curve:	Quantifies overall diagnostic discrimination as	Models tuned solely for accuracy may exhibit poor

Function / Method	1. What does it do mathematically / computationally?	2. Why use it on the Wisconsin dataset?	3. What goes wrong if omitted or misplaced?
	$\int_0^1 TPR(FPR)\, d(FPR)$.	a single threshold-independent score (0.5 to 1.0).	class separation under different operating thresholds.
`precision_recall_curve()`	Computes (Precision, Recall) coordinate pairs across classification thresholds.	Measures performance on the positive (malignant) class without being inflated by true negatives.	ROC curves can present an overly optimistic view when true negatives vastly outnumber positive cases.
`PrecisionRecallDisplay()`	Renders the Precision-Recall curve alongside the Average Precision (AP) summary score.	Visualizes the precision-recall trade-off specifically for malignant tumor detection.	Tradeoffs between precision and recall cannot be visually assessed across thresholds without this plot.
`sns.barplot()` / `plt.bar()`	Draws discrete rectangular bars with lengths proportional to given numerical values.	Visualizes feature importance scores and model comparison metrics side by side.	Tabular metrics without visual rankings make it harder to communicate relative feature contributions.